FoReco Package Demonstration with COVID-19 Hospitalizations

```
library(FoReco)    # -> To perform reconciliation
library(forecast)  # -> To obtain base forecasts
library(thief)
library(dplyr)
library(lubridate)
library(ggplot2)
```

Load and format truth data

Example with Covid-19 inc hosp data. We adjust the top aggregation level to be 8 weeks, as we can't reasonably predict anything with a longer horizon well-enough.

```
full_hosp_truth <- covidHubUtils::load_truth("HealthData", "inc hosp",
                                             temporal_resolution = "weekly",
                                             data_location = "remote_hub_repo")

# Must have time series that is a multiple of the largest aggregation level
start_date <- as.Date("2020-08-24") # a monday
end_date <- as.Date("2021-02-07")
#end_date <- as.Date("2021-07-25")

hosp_truth <- full_hosp_truth |>
  dplyr::filter(target_end_date >= start_date,
                target_end_date <= end_date,
                location == "US") |>
  mutate(day = wday(target_end_date), epi_week = epiweek(target_end_date)) |>
  dplyr::select(value, epi_week, day)

te_agg <- c(56, 28, 14, 7, 1)
  # te_agg <- tetools(56)$set
te_agg.ascending <- c(1, 7, 14, 28, 56)
  # note the wrong order creates strange-looking forecasts
# y = hosp_truth$value
freq <- max(te_agg)
```

Construct aggregates (from daily data) based on 8-week periods of epiweeks

```
set.seed(1234)

start_period <- 4
end_period <- start_period + as.numeric(end_date - start_date + 1) / freq - 1
hosp_ts <- ts(hosp_truth$value, start = c(start_period, 1), end = c(end_period, 56), frequency = 56)
hosp_agg <- aggts(hosp_ts, te_agg, align = "end", rm_na = TRUE)
agg_names <- c("8-weekly", "4-weekly", "2-weekly", "weekly", "daily")
```

```
agg_names.ascending <- c("daily", "weekly", "2-weekly", "4-weekly", "8-weekly")
hosp_agg <- setNames(hosp_agg, agg_names.ascending)
hosp_data <- list(); for (i in 1:5) hosp_data[[i]] <- hosp_agg[[6-i]]
  # To make aggregates list decent in instead of ascending
```

Plot target data

```
par(mfrow = c(3, 2))
for (i in 5:1) plot.ts(hosp_agg[[i]], plot.type="single", main=names(hosp_agg)[i])
```

Prepare truth data for plotting with forecasts

Extend truth data to end of forecasts

```
extended_truth <- full_hosp_truth |>
  dplyr::filter(target_end_date >= end_date,
                target_end_date <= end_date + weeks(8),
                location == "US") |>
  dplyr::mutate(day = wday(target_end_date), epi_week = epiweek(target_end_date)) |>
  dplyr::select("value", "epi_week", "day")

long_truth <- rbind(hosp_truth, extended_truth)
long_ts <- ts(long_truth$value, start=c(start_period, 1), end=c(end_period + 1, 56), frequency=56)

long_aggs <- aggts(long_ts, te_agg, align = "end", rm_na = TRUE)
agg_names.ascending <- c("daily", "weekly", "2-weekly", "4-weekly", "8-weekly")
long_data <- list(); for (i in 1:5) long_data[[i]] <- long_aggs[[6-i]]
```

Prepare extended truth data for plotting

```
aggregation_list <- as.list(te_agg); value <- NULL
date_list <- list(); date <- NULL
level_list <- list(); level <- NULL
frequency_list <- list(); frequency <- NULL
for (i in 1:length(te_agg)) {
  date_list[[i]] <- (1:length(long_data[[i]]))*te_agg[i]
  date <- c(date, date_list[[i]])
  value <- c(value, long_data[[i]])
  level_list[[i]] <- rep(agg_names[i], length(long_data[[i]]))
  level <- c(level, level_list[[i]])
}

extended_df <-
  tibble::tibble(date, value, level) |>
  dplyr::mutate(
    date=start_date + date-1,
    level=factor(level, levels=unique(level), ordered=TRUE)
  )
```

```
ggplot(extended_df, aes(x = date, group = level)) +
  geom_point(aes(y = value), col = 1) +
```

```
  facet_grid(rows = vars(level), scales = "free") +
  xlab("Date") + ylab(" ") +
#  theme(axis.title.x="Date", axis.title.y="") +
  ggtitle("Reconcile Forecasts New")
```

FoReco Method

Probabilistic forecasts are viewed as mean (point) forecasts, plus sampled residuals?

Calculate model fits and base point forecasts

```
fit <- lapply(hosp_agg, function(x) auto.arima(ts(x, frequency = frequency(x)))) # arima model fit
fit.arima <- list(); for (i in 1:5) fit.arima[[i]] <- fit[[6-i]] # order top down
forecast_obj.arima <- lapply(fit.arima, function(tsfit) # make into a forecast object
  forecast(tsfit, h=frequency(tsfit$x)))    # (frequency taken from inputs)
  # Could also use a for loop over `te_agg` instead

base_mean.arima <- sapply(forecast_obj.arima, function(x) x$mean)  # base mean for each level
#str(base_mean.arima, give.attr = FALSE)
res.arima <- Reduce("c", sapply(forecast_obj.arima, residuals, type='response'))
  # in-sample residuals (one-step)
#str(res.arima, give.attr = FALSE)
```

Non-parametric (joint block bootstrap) reconciliation approach

```
B <- 1000
base_mean_vec.arima <- unlist(base_mean.arima, use.names = FALSE)
res_vec.arima <- unlist(res.arima, use.names = FALSE)

base_tejb.arima <- teboot(fit.arima, B, te_agg)$sample
dim(base_tejb.arima)
#> [1] 1000   71
```

```
base_boot <- apply(base_tejb.arima, 2, quantile, na.rm = TRUE,
                   probs = c(0.025, 0.25, 0.5, 0.75, 0.975))

reco_tejb.arima <- t(apply(base_tejb.arima, 1, FoReco::terec, agg_order = te_agg,
                           res = res_vec.arima, nn = "sntz", comb = "wlsv", approach = "proj"))
  # we reconcile each member of a sample from the incoherent distribution.
  # negative numbers = set negative to 0
# str(reco_tejb, give.attr = FALSE)

# extract quantiles
boot_quantiles <- apply(reco_tejb.arima, 2, quantile, na.rm = TRUE,
                        probs = c(0.01, 0.025, seq(0.05, 0.95, 0.05), 0.975, 0.99))
boot_subset <- apply(reco_tejb.arima, 2, quantile, na.rm = TRUE,
                     probs = c(0.025, 0.25, 0.5, 0.75, 0.975))
```

Prepare bootstrap forecasts for plotting

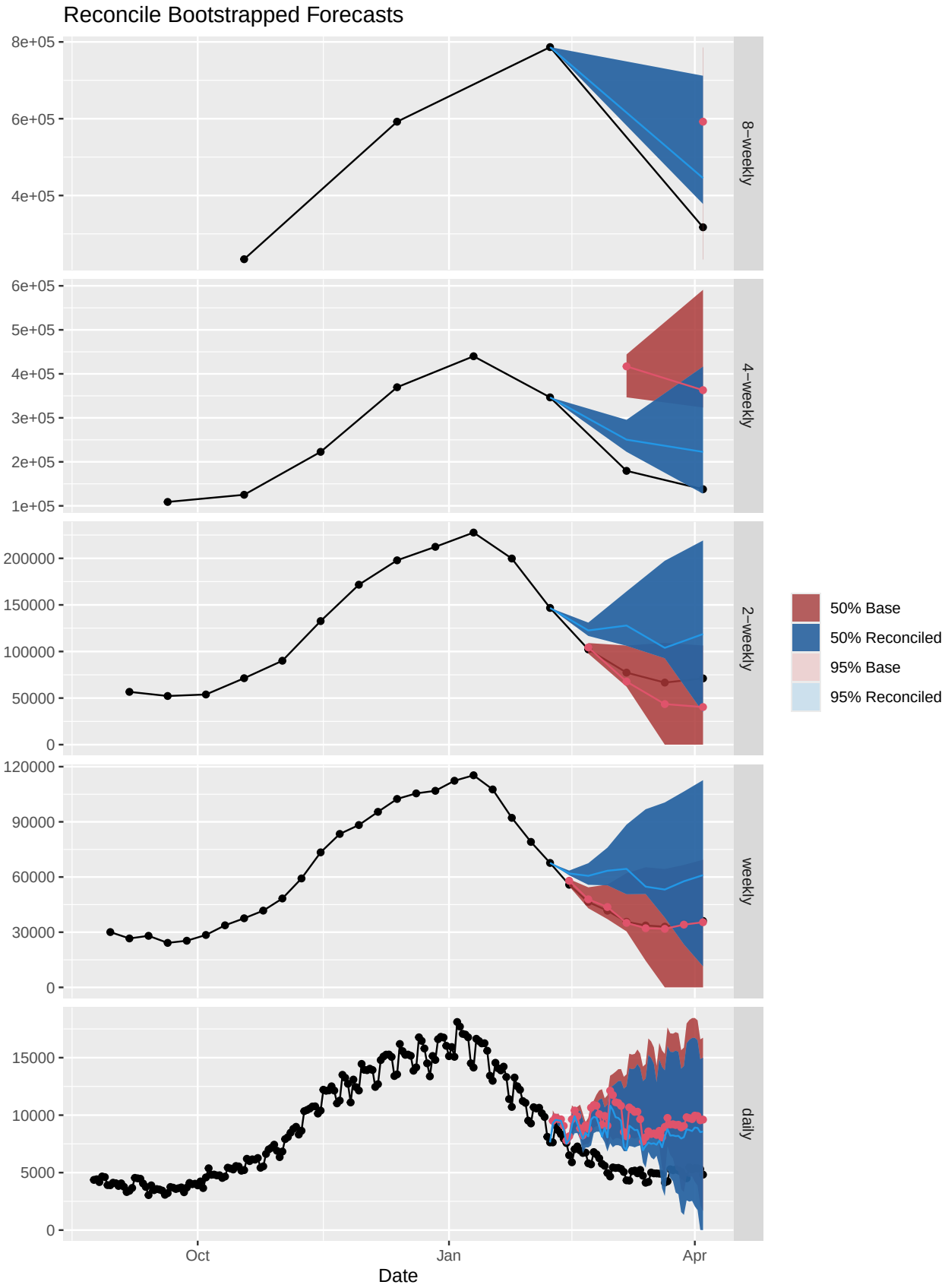
```
colnames(base_boot) <- colnames(boot_subset)
base_boot_df <- base_boot |>
  as.table() |>
  as.data.frame() |>
  dplyr::mutate(
    Var1 = paste0("q", stringr::str_remove_all(Var1, "%")),
    Var2 = as.character(Var2),
    Freq = ifelse(Freq < 0, 0, Freq)
  ) |>
  tidyr::pivot_wider(names_from = "Var1", values_from = "Freq") |>
  dplyr::mutate(
    k = as.numeric(stringr::str_extract(Var2, "\\d*(?=\\sh)")), #grab digits before h
    h = as.numeric(stringr::str_extract(Var2, "(?<=h-)\\d*")), #grab digits after h
    level = case_when(k == 56 ~ "8-weekly", k == 28 ~ "4-weekly",
                      k == 14 ~ "2-weekly", k == 7 ~ "weekly",
                      k == 1 ~ "daily", .default = NA),
    level = factor(level, levels = unique(level), ordered = TRUE),
    date = days(k*h) + end_date
  ) |>
  dplyr::select("date", "level", "q2.5":"q97.5")

bootstrap_df <- boot_subset |>
  as.table() |>
  as.data.frame() |>
  dplyr::mutate(
    Var1 = paste0("q", stringr::str_remove_all(Var1, "%")),
    Var2 = as.character(Var2),
    Freq = ifelse(Freq < 0, 0, Freq)
  ) |>
  tidyr::pivot_wider(names_from = "Var1", values_from = "Freq") |>
  dplyr::mutate(
    k = as.numeric(stringr::str_extract(Var2, "\\d*(?=\\sh)")), #grab digits before h
    h = as.numeric(stringr::str_extract(Var2, "(?<=h-)\\d*")), #grab digits after h
    level = case_when(k == 56 ~ "8-weekly", k == 28 ~ "4-weekly",
                      k == 14 ~ "2-weekly", k == 7 ~ "weekly",
                      k == 1 ~ "daily", .default = NA),
    level=factor(level, levels=unique(level), ordered=TRUE),
    date = days(k*h) + end_date
  ) |>
  dplyr::select("date", "level", "q2.5":"q97.5")
```

Plot bootstrap forecasts

```
extended_df |>
  dplyr::filter(date == end_date) |>
  dplyr::mutate(q2.5 = value, q25 = value, q50 = value, q75 = value, q97.5 = value) |>
  dplyr::select("date", "level", "q2.5": "q97.5") |>
  dplyr::bind_rows(bootstrap_df) |>
  ggplot(aes(x = date, group = level)) +
  geom_line(data = extended_df, aes(x = date, y = value, group = level), col = 1) +
  geom_point(data = extended_df, aes(x = date, y = value, group = level), col = 1) +
  geom_ribbon(data = base_boot_df, aes(x = date, ymin = q2.5, ymax = q97.5, fill = "95% Base"), alpha =
```

```
  geom_ribbon(data = base_boot_df, aes(x = date, ymin = q25, ymax = q75, fill = "50% Base"), alpha = .6
  geom_ribbon(aes(ymin = q2.5, ymax = q97.5, fill = "95% Reconciled"), alpha = .75) +
  geom_ribbon(aes(ymin = q25, ymax = q75, fill = "50% Reconciled"), alpha = .75) +
  geom_line(data = base_boot_df, aes(x = date, y = q50), col = 2) +
  geom_point(data = base_boot_df, aes(x = date, y = q50), col = 2) +
  geom_line(aes(y = q50), col = 4) +
#  geom_point(aes(y = q50), col = 4) +
  facet_grid(rows = vars(level), scales = "free") +
  scale_fill_manual(name = "", values = c("50% Reconciled" = "#00458F", "95% Reconciled" = "#C2DDEE", "
  xlab("Date") + ylab(" ") +
#  theme(axis.title.x="Date", axis.title.y="") +
  ggtitle("Reconcile Bootstrapped Forecasts")
```


Reconcile Bootstrapped Forecasts
8e+05
6e+05
4e+05
8−weekly
6e+05
5e+05
4e+05
3e+05
2e+05
1e+05
4−weekly
200000
150000
100000
50000
0
2−weekly
120000
90000
60000
30000
0
weekly
15000
10000
5000
0
daily
Oct
Jan
Apr
Date
50% Base
50% Reconciled
95% Base
95% Reconciled

Gaussian reconciliation approach

Multi-step residuals (for approximating base forecast error, which informs weights during reconciliation)

```
hres_list.arima <- lapply(fit.arima, function(mod) # mod = model within the fit object
lapply(1:frequency(mod$x), function(h)     # get frequency of ts at each agg level
  residuals(mod, type='response', h = h))) # get residuals at each level
    # note residuals = y_t - yhat_t, where t < h (aka observed time before predicted horizons)
  # list of residuals separated by aggregation level and time point
hres.arima <- Reduce("c", lapply(hres_list.arima, arrange_hres))
  # Reduce list down into vector (length 102) of residuals following order of og data
  # original data [102 total]: annual [6]; quarterly [24]; monthly [72]

# Re-arrenge multi-step residuals in a matrix form
mres.arima <- res2matrix(hres.arima, agg_order = te_agg)
  # m = 56, k* = 1 + 2 + 4 + 8 = 15, N = 336/56 = 6
  # N(k* + m) = 6(71) = 426
```

Calculate probabilistic forecasts

```
base.arima <- MASS::mvrnorm(n = B, mu = unlist(base_mean.arima), Sigma = shrink_estim(mres.arima))
base_subset <- apply(base.arima, 2, quantile, na.rm = TRUE,
                     probs = c(0.025, 0.25, 0.5, 0.75, 0.975))

reco.arima <- t(apply(base.arima, 1, terec, agg_order = te_agg, comb = "wlsv",
                      res = unlist(res.arima), nn = "sntz")) # order of aggregates doesn't matter

# extract quantiles
reco_quantiles <- apply(reco.arima, 2, quantile, na.rm = TRUE,
                        probs = c(0.01, 0.025, seq(0.05, 0.95, 0.05), 0.975, 0.99))
reco_subset <- apply(reco.arima, 2, quantile, na.rm = TRUE,
                     probs = c(0.025, 0.25, 0.5, 0.75, 0.975))
```

Prepare Gaussian forecasts for plotting

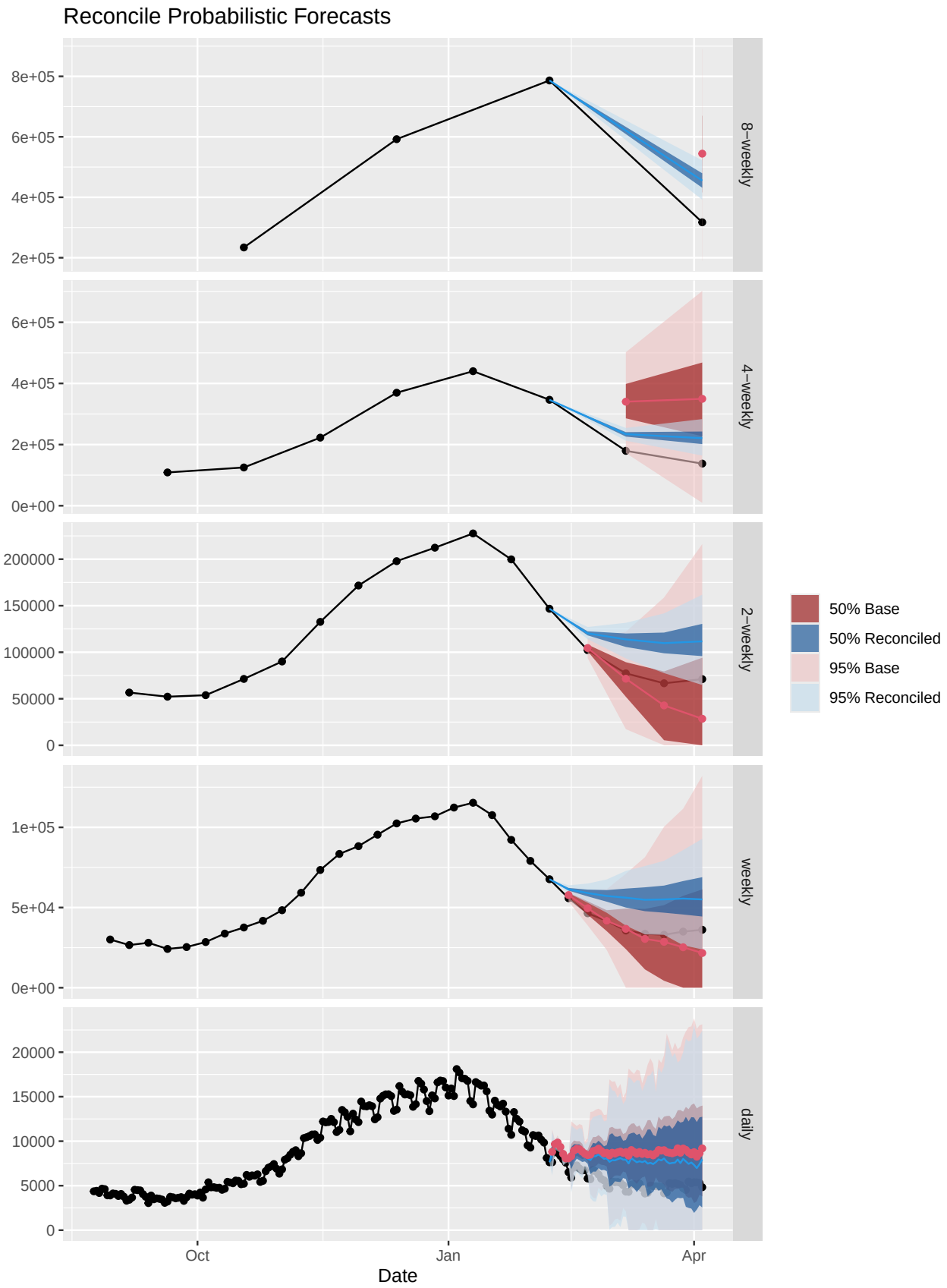
```
colnames(base_subset) <- colnames(reco_subset)
base_df <- base_subset |>
  as.table() |>
  as.data.frame() |>
  dplyr::mutate(
    Var1 = paste0("q", stringr::str_remove_all(Var1, "%")),
    Var2 = as.character(Var2),
    Freq = ifelse(Freq < 0, 0, Freq)
  ) |>
  tidyr::pivot_wider(names_from = "Var1", values_from = "Freq") |>
  dplyr::mutate(
    k = as.numeric(stringr::str_extract(Var2, "\\d*(?=\\sh)")), #grab digits before h
    h = as.numeric(stringr::str_extract(Var2, "(?<=h-)\\d*")), #grab digits after h
    level = case_when(k == 56 ~ "8-weekly", k == 28 ~ "4-weekly",
                      k == 14 ~ "2-weekly", k == 7 ~ "weekly",
                      k == 1 ~ "daily", .default = NA),
    level = factor(level, levels = unique(level), ordered = TRUE),
    date = days(k*h) + end_date
```

```
  ) |>
  dplyr::select("date", "level", "q2.5":"q97.5")

reconciled_df <- reco_subset |>
  as.table() |>
  as.data.frame() |>
  dplyr::mutate(
    Var1 = paste0("q", stringr::str_remove_all(Var1, "%")),
    Var2 = as.character(Var2),
    Freq = ifelse(Freq < 0, 0, Freq)
  ) |>
  tidyr::pivot_wider(names_from = "Var1", values_from = "Freq") |>
  dplyr::mutate(
    k = as.numeric(stringr::str_extract(Var2, "\\d*(?=\\sh)")), #grab digits before h
    h = as.numeric(stringr::str_extract(Var2, "(?<=h-)\\d*")), #grab digits after h
    level = case_when(k == 56 ~ "8-weekly", k == 28 ~ "4-weekly",
                      k == 14 ~ "2-weekly", k == 7 ~ "weekly",
                      k == 1 ~ "daily", .default = NA),
    level = factor(level, levels = unique(level), ordered = TRUE),
    date = days(k*h) + end_date
  ) |>
  dplyr::select("date", "level", "q2.5":"q97.5")
```

Plot Gaussian probabilistic forecasts

```
extended_df |>
  dplyr::filter(date == end_date) |>
  dplyr::mutate(q2.5 = value, q25 = value, q50 = value, q75 = value, q97.5 = value) |>
  dplyr::select("date", "level", "q2.5": "q97.5") |>
  dplyr::bind_rows(reconciled_df) |>
  ggplot(aes(x = date, group = level)) +
  geom_line(data = extended_df, aes(x = date, y = value, group = level), col = 1) +
  geom_point(data = extended_df, aes(x = date, y = value, group = level), col = 1) +
  geom_ribbon(data = base_df, aes(x = date, ymin = q2.5, ymax = q97.5, fill = "95% Base"), alpha = .6) +
  geom_ribbon(data = base_df, aes(x = date, ymin = q25, ymax = q75, fill = "50% Base"), alpha = .6) +
  geom_ribbon(aes(ymin = q2.5, ymax = q97.5, fill = "95% Reconciled"), alpha = .6) +
  geom_ribbon(aes(ymin = q25, ymax = q75, fill = "50% Reconciled"), alpha = .6) +
  geom_line(aes(y = q50), col = 4) +
#  geom_point(aes(y = q50), col = 4) +
  geom_line(data = base_df, aes(x = date, y = q50), col = 2) +
  geom_point(data = base_df, aes(x = date, y = q50), col = 2) +
  facet_grid(rows = vars(level), scales = "free") +
  scale_fill_manual(name = "", values = c("50% Reconciled" = "#00458F", "95% Reconciled" = "#C2DDEE", "
  xlab("Date") + ylab(" ") +
#  theme(axis.title.x="Date", axis.title.y="") +
  ggtitle("Reconcile Probabilistic Forecasts")
```


Reconcile Probabilistic Forecasts
8e+05
6e+05
4e+05
2e+05
8−weekly
6e+05
4e+05
2e+05
0e+00
4−weekly
200000
150000
100000
50000
0
2−weekly
1e+05
5e+04
0e+00
weekly
20000
15000
10000
5000
0
daily
Oct
Jan
Apr
Date
50% Base
50% Reconciled
95% Base
95% Reconciled

`thief` Method

Aggregate target data

```
old_hosp_agg <- tsaggregates(hosp_ts, m = 56, aggregatelist = as.list(te_agg))
  # same as hosp_agg EXCEPT has separate x and y components for base r plotting
for(i in seq_along(old_hosp_agg)) names(old_hosp_agg)[[i]] <- agg_names.ascending[i]
```

```
plot(old_hosp_agg, main = "Covid-19 Inc Hosp")
```

Compute base forecasts

```
base_fc_day <- list()
for(i in seq_along(old_hosp_agg))
  base_fc_day[[i]] <- forecast(auto.arima(old_hosp_agg[[i]]), h=frequency(old_hosp_agg[[i]]),
                               level = c(50, 95)) # has 50% interval instead of default 80%
```

Reconcile forecasts

```
reconciled_fc_day <- reconcilethief(base_fc_day, aggregatelist = as.list(te_agg))
```

Prepare old reconciled forecasts for plotting

```
aggregation_list <- as.list(te_agg); date_list <- list(); frequency <- 56
q2.5 <- NULL; q25 <- NULL; q50 <- NULL; q75 <- NULL; q97.5 <- NULL
level_list <- list(); level <- NULL
for (i in 1:length(aggregation_list)) {
  date_list[[i]] <- (length(old_hosp_agg[[1+length(aggregation_list) - i]]) +
    (1:(frequency/aggregation_list[[i]])))*aggregation_list[[i]]
  date <- c(date, date_list[[i]])
  q2.5 <- c(q2.5, base_fc_day[[1+length(aggregation_list) - i]][["lower"]][,2])
  q25 <- c(q25, base_fc_day[[1+length(aggregation_list) - i]][["lower"]][,1])
  q50 <- c(q50, base_fc_day[[1+length(aggregation_list) - i]][["mean"]])
  q75 <- c(q75, base_fc_day[[1+length(aggregation_list) - i]][["upper"]][,1])
  q97.5 <- c(q97.5, base_fc_day[[1+length(aggregation_list) - i]][["upper"]][,2])
  level_list[[i]] <- rep(agg_names.ascending[1+length(aggregation_list) - i],
                         frequency/aggregation_list[[i]])
  level <- c(level, level_list[[i]])
}

base_df_old <-
  tibble::tibble(q2.5, q25, q50, q75, q97.5, level) |>
  cbind(date = c(56, 28, 56, seq(14, 56, 14), seq(7, 56, 7), 1:56)) |>
  dplyr::mutate(
    date = days(date) + end_date,
    level=factor(level, levels=unique(level), ordered=TRUE),
    across(where(is.numeric), function(x) ifelse(x < 0, 0, x)),
```

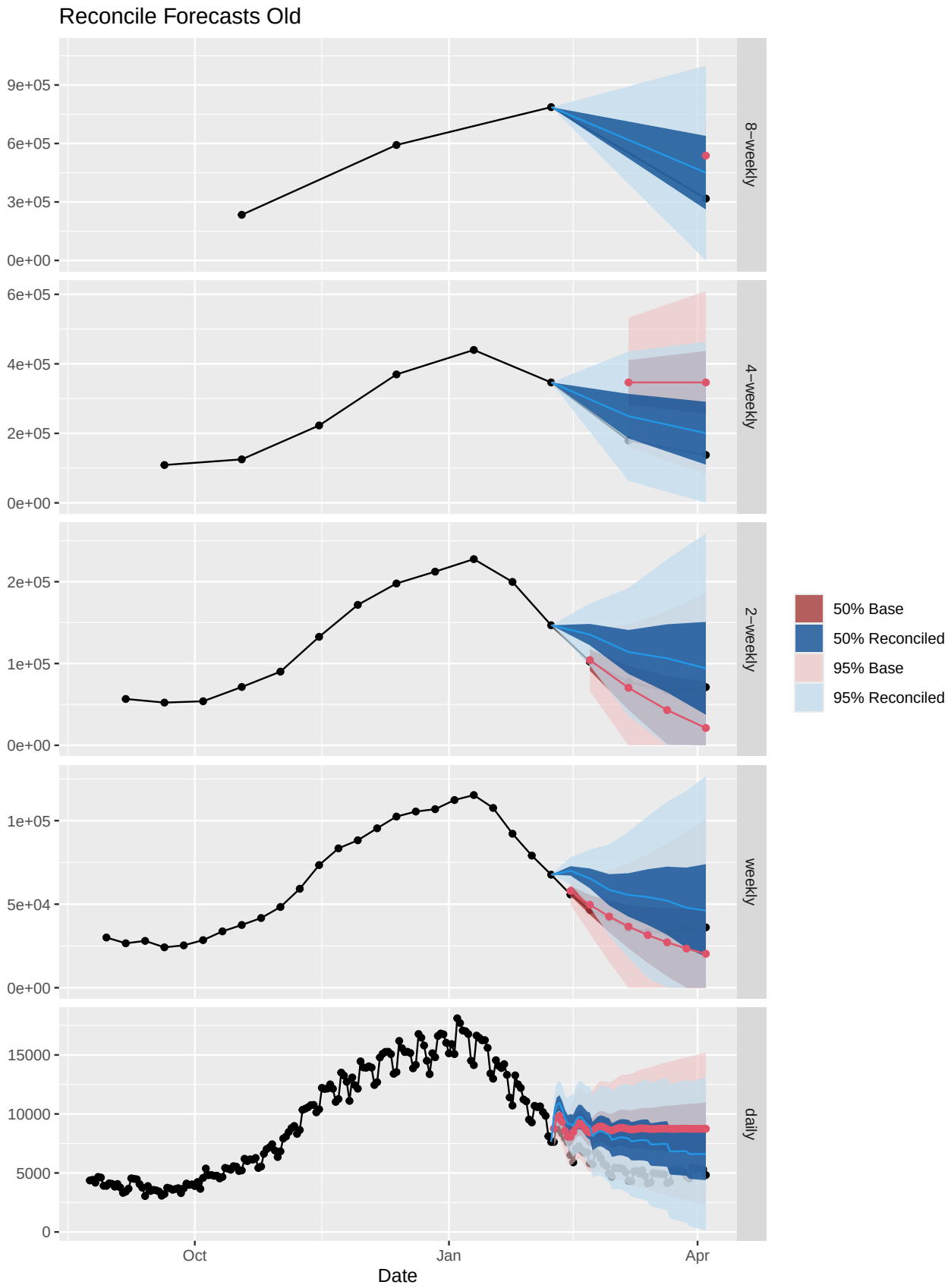
```
    .before = 1
)

aggregation_list <- as.list(te_agg); date_list <- list(); frequency <- 56
q2.5 <- NULL; q25 <- NULL; q50 <- NULL; q75 <- NULL; q97.5 <- NULL
level_list <- list(); level <- NULL
for (i in 1:length(aggregation_list)) {
  date_list[[i]] <- (length(old_hosp_agg[[1+length(aggregation_list) - i]]) +
    (1:(frequency/aggregation_list[[i]])))*aggregation_list[[i]]
  date <- c(date, date_list[[i]])
  q2.5 <- c(q2.5, reconciled_fc_day[[1+length(aggregation_list) - i]][["lower"]][,2])
  q25 <- c(q25, reconciled_fc_day[[1+length(aggregation_list) - i]][["lower"]][,1])
  q50 <- c(q50, reconciled_fc_day[[1+length(aggregation_list) - i]][["mean"]])
  q75 <- c(q75, reconciled_fc_day[[1+length(aggregation_list) - i]][["upper"]][,1])
  q97.5 <- c(q97.5, reconciled_fc_day[[1+length(aggregation_list) - i]][["upper"]][,2])
  level_list[[i]] <- rep(agg_names.ascending[1+length(aggregation_list) - i],
                         frequency/aggregation_list[[i]])
  level <- c(level, level_list[[i]])
}

reconciled_df_old <-
  tibble::tibble(q2.5, q25, q50, q75, q97.5, level) |>
  cbind(date = c(56, 28, 56, seq(14, 56, 14), seq(7, 56, 7), 1:56)) |>
  dplyr::mutate(
    date = days(date) + end_date,
    level=factor(level, levels=unique(level), ordered=TRUE),
    across(where(is.numeric), function(x) ifelse(x < 0, 0, x)),
    .before = 1
  )
```

Plot old reconciled forecasts

```
extended_df |>
  dplyr::filter(date == end_date) |>
  dplyr::mutate(q2.5 = value, q25 = value, q50 = value, q75 = value, q97.5 = value) |>
  dplyr::select("date", "level", "q2.5": "q97.5") |>
  dplyr::bind_rows(reconciled_df_old) |>
  ggplot(aes(x = date, group = level)) +
  geom_line(data = extended_df, aes(x = date, y = value, group = level), col = 1) +
  geom_point(data = extended_df, aes(x = date, y = value, group = level), col = 1) +
  geom_ribbon(data = base_df_old, aes(x = date, ymin = q2.5, ymax = q97.5, fill = "95% Base"), alpha =
  geom_ribbon(data = base_df_old, aes(x = date, ymin = q25, ymax = q75, fill = "50% Base"), alpha = .6)
  geom_ribbon(aes(ymin = q2.5, ymax = q97.5, fill = "95% Reconciled"), alpha = .75) +
  geom_ribbon(aes(ymin = q25, ymax = q75, fill = "50% Reconciled"), alpha = .75) +
  geom_line(data = base_df_old, aes(x = date, y = q50), col = 2) +
  geom_point(data = base_df_old, aes(x = date, y = q50), col = 2) +
  geom_line(aes(y = q50), col = 4) +
#  geom_point(aes(y = q50), col = 4) +
  facet_grid(rows = vars(level), scales = "free") +
  scale_fill_manual(name = "", values = c("50% Reconciled" = "#00458F", "95% Reconciled" = "#C2DDEE", "
  xlab("Date") + ylab(" ") +
#  theme(axis.title.x="Date", axis.title.y="") +
  ggtitle("Reconcile Forecasts Old")
```


Reconcile Forecasts Old
8−weekly
4−weekly
2−weekly
weekly
daily
9e+05
6e+05
3e+05
0e+00
4e+05
2e+05
1e+05
5e+04
15000
10000
5000
0
Oct
Jan
Apr
Date
50% Base
50% Reconciled
95% Base
95% Reconciled

Conclusion

Overall, the Gaussian probabilistic method and THieF method tend to produce fairly similar results. There is some difference in terms of interval width, with the Gaussian probabilistic forecasts producing wider intervals for lower temporal levels in comparison to the THieF methodology and narrower intervals for higher temporal levels. The non-parametric bootstrap method tends to be less accurate and produces highly variable intervals based on the sampling, which reflects the findings in the `FoReco` paper. This method will sometimes produce strange results, especially when the number of samples is low.

Note that all these methods begin with the same point forecasts and residuals but obtain the finalized probabilistic forecasts in different ways:

1. **Non-parametric Bootstrap (probabilistic):** Bootstrap the original point forecasts using uncertainty estimated by their residuals, perform reconciliation of the samples, then calculate quantiles from the samples.
2. **Gaussian Probabilistic:** Calculate a normal distribution from which to draw the base forecast samples using the original point forecasts and residuals as the mean and variance estimates respectively. Perform reconciliation of the samples, then calculate quantiles from the samples.
3. **THieF:** Calculate the desired quantiles from the original base forecasts by assuming a normal distribution sort, likely using the residuals as a variance estimate. Perform reconciliation for each of the quantiles separately by treating them like point forecasts.

Based on these preliminary results, I'm also curious about whether implementing a 7 day rolling average for the daily forecasts may be useful in improving accuracy.